

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN-TIN

DỰ BÁO RỦI RO VỠ NỢ TÍN DỤNG BẰNG MACHINE LEARNING

Loan Default Prediction Using Machine Learning

ĐỒ ÁN I — MACHINE LEARNING ỨNG DỤNG

Sinh viên thực hiện: Đoàn Danh Long

Mã số sinh viên: 20237354

Giảng viên hướng dẫn: Nguyễn Cảnh Nam

Học kỳ: 2025.2 — Năm học 2025–2026

Hà Nội, 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến Giảng viên hướng dẫn — Thầy Nguyễn Cảnh Nam, Khoa Toán–Tin, Trường Đại học Bách Khoa Hà Nội — đã tận tình hướng dẫn, góp ý và định hướng trong suốt quá trình thực hiện đồ án.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện thuận lợi để em có thể hoàn thành đồ án đúng tiến độ.

Dù đã cố gắng hết sức, báo cáo chắc chắn còn nhiều thiếu sót. Em rất mong nhận được nhận xét và góp ý từ Thầy để tiếp tục cải thiện trong những nghiên cứu tiếp theo.

Hà Nội, tháng 4 năm 2026

Đoàn Danh Long

TÓM TẮT

Báo cáo trình bày nghiên cứu ứng dụng Machine Learning vào bài toán dự báo rủi ro vỡ nợ tín dụng, sử dụng bộ dữ liệu *Give Me Some Credit* (Kaggle, 150.000 hồ sơ vay). Bốn mô hình Supervised Learning được xây dựng và đánh giá theo thứ tự tăng dần độ phức tạp: Logistic Regression (AUC = 0,8432), Decision Tree CART (AUC = 0,8579), Random Forest (AUC = 0,8703), và XGBoost (AUC = 0,8714). Mô hình tốt nhất — XGBoost với cơ chế boosting bậc hai — vượt mục tiêu đặt ra (AUC > 0,87) và được phân tích sâu thông qua SHAP Tree-Explainer. Feature Engineering có chủ đích (TotalDelinquencyScore, FinancialStressIndex) được SHAP xác nhận là 2 trong top-3 đặc trưng quan trọng nhất. Ngưỡng tối ưu theo F_2 -score là $t = 0,625$, tăng Recall từ 51,6% lên 66,9%, giảm chi phí kinh doanh ước tính 2 triệu USD trên tập kiểm tra. Sản phẩm cuối là ứng dụng Streamlit tương tác, giải thích từng quyết định tín dụng bằng SHAP waterfall chart.

Từ khóa: dự báo vỡ nợ, chấm điểm tín dụng, XGBoost, SHAP, tối ưu F_β , class imbalance, rủi ro tín dụng.

Contents

LỜI CẢM ƠN	1
TÓM TẮT	2
1 GIỚI THIỆU	6
1.1 Tính cấp thiết của vấn đề	6
1.2 Mục tiêu nghiên cứu	6
1.3 Phạm vi nghiên cứu	7
2 CƠ SỞ LÝ THUYẾT	8
2.1 Bài toán Binary Classification	8
2.1.1 Định nghĩa hình thức	8
2.1.2 Class Imbalance và hệ quả	8
2.2 Evaluation Metrics	9
2.3 Logistic Regression	9
2.3.1 Hypothesis Function và log-odds	9
2.3.2 Ước lượng tham số — Maximum Likelihood Estimation	10
2.3.3 Regularization	10
2.4 Decision Tree CART	10
2.4.1 Tiêu chí phân chia — Gini Impurity	10
2.4.2 Kiểm soát Overfitting	11
2.4.3 <code>class_weight = 'balanced'</code> (Bù đắp Class Imbalance)	11
2.5 Random Forest	11
2.5.1 Bootstrap Aggregating (Bagging)	11
2.5.2 Giảm Variance — Phân tích lý thuyết	11
2.5.3 Feature Importance	12
2.6 XGBoost — Thuật toán Gradient Boosting Bậc hai	12
2.6.1 Mô hình Additive	12
2.6.2 Objective Function và xấp xỉ Taylor bậc hai	12
2.6.3 Trọng số lá tối ưu dạng nghiệm đóng	13
2.6.4 Xử lý Class Imbalance — <code>scale_pos_weight</code>	13
2.6.5 SHAP — Shapley Additive exPlanations	13
2.7 Nghiên cứu Liên quan	14
3 DỮ LIỆU VÀ TIỀN XỬ LÝ	15
3.1 Mô tả Dataset	15

3.1.1	Tổng quan	15
3.1.2	Ý nghĩa tài chính từng đặc trưng	16
3.1.3	Phân phối và outliers	16
3.2	Phân tích EDA	17
3.2.1	Mối quan hệ phi tuyến giữa <code>RevolvingUtilization</code> và tỷ lệ vỡ nợ	17
3.2.2	Spearman Correlation giữa các biến delinquency	17
3.2.3	Tỷ lệ vỡ nợ theo số lần trễ hạn	17
3.3	Xử lý Missing Values	18
3.3.1	Kiểm định cơ chế khuyết thiếu (missing mechanism)	18
3.3.2	Lý do chọn KNN Imputer	18
3.4	Feature Engineering	19
3.4.1	<code>TotalDelinquencyScore</code>	19
3.4.2	<code>FinancialStressIndex</code>	19
3.4.3	<code>AbsoluteDebt</code> — Dư nợ tuyệt đối (tên code: <code>AbsoluteMonthlyDebt</code>)	20
3.4.4	<code>DelinquencyTrend</code> (Cán cân mức độ trễ hạn)	20
3.5	Xử lý Class Imbalance	21
3.5.1	Chiến lược đã thử	21
3.5.2	Quyết định cuối cùng	21
3.6	Phân chia Dữ liệu	21
4	THỰC NGHIỆM VÀ ĐÁNH GIÁ	23
4.1	Thiết lập Thực nghiệm	23
4.2	Logistic Regression	23
4.2.1	Cấu hình và kết quả	23
4.2.2	Phân tích Odds Ratios	24
4.3	Decision Tree CART	24
4.3.1	Cấu hình và kết quả	24
4.3.2	Trực quan hóa Cây (3 tầng đầu)	24
4.4	Random Forest	25
4.4.1	Cấu hình và kết quả	25
4.5	XGBoost	25
4.5.1	Cấu hình và kết quả	25
4.5.2	Phân tích SHAP	26
4.6	Bảng So sánh Tổng hợp	28
4.6.1	Kiểm định Thống kê Sự khác biệt AUC — DeLong Test	29
4.7	Phân tích Sai số	30
4.7.1	Cấu trúc lỗi tại ngưỡng = 0,77	30
4.7.2	Đặc điểm nhóm False Negative	30
4.7.3	Profile của False Positives	31

4.7.4	Tối ưu Ngưỡng — Điểm số F_β	31
4.7.5	Learning Curve — Chẩn đoán Bias-Variance	32
4.8	Bàn luận 3 Tranh luận Lớn	33
4.8.1	Khả năng Giải thích so với Hiệu suất	33
4.8.2	Hướng Tiếp cận Dữ liệu so với Mô hình	33
4.8.3	Accuracy so với Recall — Chiến lược Ngưỡng	34
4.9	Probability Calibration	34
4.9.1	Động cơ	34
4.9.2	Brier Score và Brier Skill Score	34
4.9.3	Reliability Diagram	35
4.9.4	Post-hoc Calibration và Hạn chế	35
5	SẢN PHẨM — STREAMLIT DASHBOARD	36
5.1	Kiến trúc Hệ thống	36
5.2	Đặc trưng Đầu vào	37
5.3	Tính năng chính	38
5.4	Các Tình huống Minh họa	38
5.4.1	Hồ sơ 1 — Khách hàng an toàn	38
5.4.2	Hồ sơ 2 — Khách hàng rủi ro cao	39
5.4.3	Hồ sơ 3 — Khách hàng ranh giới	39
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	40
6.1	Tóm tắt Kết quả	40
6.2	Hạn chế	41
6.3	Hướng Phát triển	41
A	PHỤ LỤC	44
A.1	Cấu trúc Thư mục	44
A.2	Công thức Tổng hợp	44
A.3	Cài đặt Môi trường	45
A.4	Hướng dẫn Reproduce Kết quả	45

1. GIỚI THIỆU

1.1 Tính cấp thiết của vấn đề

Rủi ro tín dụng là một trong những nguy cơ lớn nhất đối với sự ổn định của hệ thống tài chính. Theo Ngân hàng Nhà nước Việt Nam, tỷ lệ nợ xấu (NPL — Non-Performing Loan) toàn hệ thống cuối năm 2023 ở mức 4,55%, tương đương hàng trăm nghìn tỷ đồng tài sản có nguy cơ mất vốn [1]. Trên phạm vi quốc tế, cuộc khủng hoảng tài chính toàn cầu 2008 có nguồn gốc trực tiếp từ việc định giá sai rủi ro tín dụng trong thị trường thế chấp bất động sản Mỹ.

Quy trình thẩm định tín dụng truyền thống dựa vào phán đoán của chuyên viên tín dụng, sàng lọc thủ công một số chỉ số tài chính. Phương pháp này bộc lộ hai yếu điểm căn bản: **(1)** không mở rộng được quy mô khi khối lượng đơn vay tăng đột biến (đặc biệt trong bối cảnh ngân hàng số), và **(2)** thiếu tính nhất quán — cùng một hồ sơ có thể được đánh giá khác nhau bởi hai chuyên viên. Các mô hình tính điểm tín dụng (credit scoring) thống kê ra đời từ những năm 1950 (FICO Score, 1956) giải quyết phần nào vấn đề quy mô, nhưng tính tuyến tính của chúng bỏ qua nhiều tương tác phi tuyến giữa các biến rủi ro.

Sự phát triển của Machine Learning mở ra khả năng xây dựng mô hình chấm điểm tín dụng tự động, học trực tiếp từ lịch sử hàng triệu hồ sơ, nắm bắt được cả những tương tác phi tuyến phức tạp. Đồng thời, kỹ thuật SHAP (SHapley Additive exPlanations) giải quyết vấn đề “black-box” — yêu cầu pháp lý bắt buộc ngân hàng phải giải thích lý do từ chối cho khách hàng (Basel III, Điều 431).

1.2 Mục tiêu nghiên cứu

Nghiên cứu này đặt ra các mục tiêu SMART sau:

Table 1.1. Mục tiêu SMART* và kết quả đạt được

Mục tiêu	Chỉ tiêu cụ thể	Kết quả
Xây dựng mô hình Classification	AUC-ROC $\geq 0,87$ trên tập kiểm tra	0,8714
So sánh 4 thuật toán Feature Engineering	AUC, F1, P, R, thời gian huấn luyện	Bảng 4.1
có cơ sở	≥ 2 đặc trưng mới vào top-5 SHAP	2 trong top-3 SHAP
Tối ưu ngưỡng theo chi phí kinh doanh	F_2 -score, ước tính chi phí	$t = 0,625$, Recall= 66,9%
Sản phẩm demo	Ứng dụng chạy được, giải thích SHAP	Streamlit

* SMART: Specific (Cụ thể) — Measurable (Đo lường được) — Achievable (Khả thi) — Relevant (Thực tế) — Time-bound (Có thời hạn).

1.3 Phạm vi nghiên cứu

- **Dữ liệu:** Bộ dữ liệu công khai *Give Me Some Credit* (Kaggle, 2011), 149.999 hồ sơ vay tiêu dùng Mỹ.
- **Mô hình:** 4 mô hình Binary Classification có giám sát: Logistic Regression, Decision Tree CART, Random Forest, XGBoost.
- **Phạm vi không bao gồm:** Neural network sâu, dữ liệu thời gian thực, tích hợp hệ thống ngân hàng, các thị trường ngoài phạm vi bộ dữ liệu.
- **Ngôn ngữ lập trình:** Python 3.14, scikit-learn 1.8.0, XGBoost 3.2.0, SHAP 0.51.0.

2. CƠ SỞ LÝ THUYẾT

2.1 Bài toán Binary Classification

2.1.1 Định nghĩa hình thức

Cho tập dữ liệu huấn luyện $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ với $\mathbf{x}_i \in \mathbb{R}^p$ là vector đặc trưng p chiều và $y_i \in \{0, 1\}$ là nhãn lớp. Mục tiêu là học hàm $f: \mathbb{R}^p \rightarrow [0, 1]$ sao cho $f(\mathbf{x}) = P(y = 1 | \mathbf{x})$ ước lượng chính xác xác suất hậu nghiệm (posterior probability) của sự kiện vỡ nợ.

Quyết định phân loại tại ngưỡng t :

$$\hat{y} = \mathbb{I}[f(\mathbf{x}) \geq t] \quad (2.1)$$

2.1.2 Class Imbalance và hệ quả

Trong dữ liệu này, tỷ lệ vỡ nợ là 6,68%, tạo ra tỷ lệ mất cân bằng 1:14. Điều này dẫn đến hai hệ quả quan trọng:

Hệ quả 1 — Accuracy không phù hợp và không đáp ứng được yêu cầu của bài toán thực tế: Mô hình baseline $f(\mathbf{x}) \equiv 0$ (đoán tất cả không vỡ nợ) đạt Accuracy = 93,32% nhưng Recall = 0 — không có giá trị ứng dụng trong phát hiện rủi ro tín dụng.

Hệ quả 2 — Ngưỡng tối ưu $\neq 0,5$: Với $P(y = 1) = 0,0668$, quyết định Bayes tối ưu tối thiểu hóa kỳ vọng chi phí phân loại. Phân loại là 1 (từ chối vay) khi:

$$c_{FN} \cdot P(y = 1 | \mathbf{x}) > c_{FP} \cdot P(y = 0 | \mathbf{x}) \quad (2.2)$$

Rút gọn: phân loại là 1 khi $P(y = 1 | \mathbf{x}) > t^*$ với **ngưỡng Bayes tối ưu**:

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}} \quad (2.3)$$

Với $c_{FN} = \$11.250$ và $c_{FP} = \$500$: $t^* = 500/(500 + 11.250) \approx 0,043$. Lưu ý quan trọng: công thức này hoạt động trực tiếp trên *xác suất hậu nghiệm* $P(y = 1 | \mathbf{x})$, không phụ thuộc vào tỷ lệ prior $P(y = 0)/P(y = 1)$ vì prior đã được hấp thụ vào $P(y = 1 | \mathbf{x})$ thông qua định lý Bayes — điều kiện: $P(y = 1 | \mathbf{x})$ phải là xác suất hậu nghiệm calibrated từ phân phối gốc, không phải đầu ra thô của mô hình huấn luyện trên dữ liệu resampled. Tuy nhiên ngưỡng cực thấp này ($t^* = 0,043$) khiến $\sim 74\%$ hồ sơ bị gán nhãn TỪ CHỐI — quá tải hệ thống thẩm định thủ công, không khả thi trong vận hành. Điều này dẫn đến cách tiếp cận thực tế bằng F_2 -metric để tìm ngưỡng cân bằng giữa Recall và khả năng vận hành thực tế (Giai đoạn 4).

2.2 Evaluation Metrics

Confusion Matrix:

	Pred = 0	Pred = 1
Actual = 0	TN	FP
Actual = 1	FN	TP

Từ đó:

- Precision = $\frac{TP}{TP + FP}$ — Trong số những người bị từ chối, bao nhiêu % đúng là vỡ nợ?
- Recall = $\frac{TP}{TP + FN}$ — Trong số người thực sự vỡ nợ, bắt được bao nhiêu %?
- $F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 P + R}$ — Trung bình điều hòa có trọng số.

AUC-ROC đo khả năng phân biệt của mô hình, không phụ thuộc ngưỡng:

$$\text{AUC} = P(f(\mathbf{x}^+) > f(\mathbf{x}^-)) \quad (2.4)$$

với $\mathbf{x}^+$, $\mathbf{x}^-$ lần lượt là một mẫu dương và âm ngẫu nhiên.

AUC-PR (Precision-Recall curve area) nhạy hơn với lớp thiểu số, phù hợp đánh giá trong trường hợp class imbalance nghiêm trọng. Nghiên cứu này chọn tối ưu hóa AUC-ROC trong cross-validation để đảm bảo khả năng xếp hạng tổng thể (ranking ability), đồng thời dùng F_2 -score ở khâu chọn ngưỡng để ưu tiên xử lý lớp thiểu số — qua đó bù đắp hạn chế của AUC-ROC trên dữ liệu mất cân bằng mà không cần tính AUC-PR riêng.

2.3 Logistic Regression

2.3.1 Hypothesis Function và log-odds

Logistic Regression mô hình hóa xác suất hậu nghiệm qua hàm sigmoid:

$$P(y = 1 \mid \mathbf{x}; \boldsymbol{\beta}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}} \quad (2.5)$$

Log-odds (logit) là hàm tuyến tính của các đặc trưng:

$$\ln \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \mathbf{x}^\top \boldsymbol{\beta} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p \quad (2.6)$$

Ý nghĩa tài chính: e^{β_j} là *tỷ suất chênh* (odds ratio) của đặc trưng j — tăng x_j lên 1 đơn vị nhân tỷ suất chênh vỡ nợ lên e^{β_j} lần.

2.3.2 Ước lượng tham số — Maximum Likelihood Estimation

Hàm log-likelihood âm (cross-entropy loss):

$$\mathcal{L}(\beta) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln \sigma(\mathbf{x}_i^\top \beta) + (1 - y_i) \ln(1 - \sigma(\mathbf{x}_i^\top \beta))] \quad (2.7)$$

Gradient (convex function, có nghiệm toàn cục):

$$\frac{\partial \mathcal{L}}{\partial \beta} = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}), \quad \hat{y}_i = \sigma(\mathbf{x}_i^\top \beta) \quad (2.8)$$

2.3.3 Regularization

L2 (Ridge): $\mathcal{L}_{L2} = \mathcal{L}(\beta) + \frac{1}{2C} \|\beta\|_2^2$

Nghiệm bị co về 0 nhưng không bằng 0. Ổn định về số với các đặc trưng có tương quan cao.

L1 (Lasso): $\mathcal{L}_{L1} = \mathcal{L}(\beta) + \frac{1}{C} \|\beta\|_1$

Cho nghiệm thưa — tự động loại các đặc trưng không quan trọng. C là nghịch đảo cường độ regularization (nhỏ = regularization mạnh).

Trong thực nghiệm (solver SAGA), L1 với $C = 0,001$ triết tiêu hệ số `NumberOfTimes90DaysLate` về không — bằng chứng rằng `TotalDelinquencyScore` đã mã hóa đầy đủ thông tin đó.

2.4 Decision Tree CART

2.4.1 Tiêu chí phân chia — Gini Impurity

CART (Classification and Regression Trees) phân chia đệ quy không gian đặc trưng. Với node t chứa N_t mẫu, Gini impurity:

$$G(t) = 1 - \sum_{c \in \{0,1\}} p(c | t)^2 = 2p_t(1 - p_t) \quad (2.9)$$

với $p_t = P(y = 1 | t)$ là tỷ lệ dương trong node. $G(t) = 0$ khi node thuần khiết ($p_t \in \{0, 1\}$), $G(t) = 0,5$ khi node hỗn tạp tối đa ($p_t = 0,5$).

Giải thích trực quan: Gini impurity đo mức độ hỗn tạp của một node — xác suất một mẫu ngẫu nhiên bị phân loại sai nếu gán nhãn theo phân phối trong node đó. Node thuần khiết: $G = 0$; node hỗn tạp tối đa ($p_t = 0,5$): $G = 0,5$.

Tiêu chí phân chia tại node t , chọn đặc trưng j và ngưỡng θ để maximize:

$$\Delta G(t, j, \theta) = G(t) - \frac{N_L}{N_t} G(t_L) - \frac{N_R}{N_t} G(t_R) \quad (2.10)$$

2.4.2 Kiểm soát Overfitting

Cây đầy đủ chiều sâu thường gây overfitting với dữ liệu huấn luyện. Kiểm soát thông qua:

- `max_depth`: giới hạn độ sâu tối đa (qua thực nghiệm `max_depth = 10` cho kết quả tốt nhất);
- `min_samples_leaf`: số mẫu tối thiểu tại nút lá (200 — tránh các nút lá quá nhỏ, thiếu tính đại diện).

2.4.3 `class_weight = 'balanced'` (Bù đắp Class Imbalance)

Với mất cân bằng 1:14, scikit-learn gán $w_+ = N/(2 \cdot N_+) = 149.999/(2 \times 10.026) \approx 7,48$ cho lớp thiểu số và $w_- \approx 0,54$ cho lớp đa số. Tỷ lệ $w_+/w_- \approx 14$ bù đắp chính xác mức mất cân bằng. Điều chỉnh tiêu chí phân chia thành weighted Gini:

$$G_w(t) = 1 - \sum_c \left(\frac{\sum_{i:y_i=c} w_i}{\sum_i w_i} \right)^2 \quad (2.11)$$

2.5 Random Forest

2.5.1 Bootstrap Aggregating (Bagging)

Random Forest huấn luyện B cây quyết định độc lập, mỗi cây trên một bootstrap sample:

$$\hat{f}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x}) \quad (2.12)$$

Mỗi bootstrap sample lấy N mẫu có hoàn trả từ N mẫu gốc. Xác suất để một mẫu *không* được chọn là $(1 - 1/N)^N \rightarrow e^{-1} \approx 36,8\%$ — đây chính là **Out-of-Bag (OOB) samples** dùng để ước lượng sai số ngoài mẫu mà không cần tập kiểm định.

2.5.2 Giảm Variance — Phân tích lý thuyết

Cho B cây với phương sai σ^2 và hệ số tương quan ρ giữa mọi cặp cây:

$$\mathbb{V} \left(\frac{1}{B} \sum_b f_b \right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2 \quad (2.13)$$

Khi $B \rightarrow \infty$, variance $\rightarrow \rho \sigma^2$. Lấy mẫu ngẫu nhiên theo đặc trưng ($\sqrt{p}$ đặc trưng mỗi lần phân chia) giảm ρ , do đó giảm variance. Đây là lý do Random Forest vượt Decision Tree đơn về tổng quát hóa.

OOB score trong thực nghiệm: 0,8286 — ước lượng không thiên lệch của AUC trên tập kiểm tra (thực tế AUC = 0,8703, chênh lệch do OOB có xu hướng ước lượng thận trọng).

2.5.3 Feature Importance

Mean Decrease in Impurity (MDI):

$$FI(j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in f_b: \text{split on } j} \frac{N_t}{N} \cdot \Delta G(t, j) \quad (2.14)$$

2.6 XGBoost — Thuật toán Gradient Boosting Bậc hai

2.6.1 Mô hình Additive

XGBoost xây dựng additive ensemble: $F_T(\mathbf{x}) = \sum_{t=1}^T f_t(\mathbf{x})$, với f_t là cây quyết định (weak learner) thứ t . Tại bước t :

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \eta f_t(\mathbf{x}) \quad (2.15)$$

$\eta \in (0, 1)$ là learning rate (= 0,05), kiểm soát shrinkage.

2.6.2 Objective Function và xấp xỉ Taylor bậc hai

Hàm mục tiêu tại bước t :

$$\mathcal{O}^{(t)} = \sum_{i=1}^N l(y_i, F_{t-1}(\mathbf{x}_i) + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (2.16)$$

Trong đó regularization term:

$$\Omega(f_t) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2 \quad (2.17)$$

T là số lá, w_j là trọng số lá, γ là penalty tối thiểu gain để tạo phân chia, λ là L2 trên trọng số lá.

Xấp xỉ Taylor bậc hai (bỏ hạng bậc nhất vì đã biết F_{t-1}):

$$\mathcal{O}^{(t)} \approx \sum_{i=1}^N \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) \quad (2.18)$$

với gradient $g_i = \partial_{F_{t-1}} l(y_i, F_{t-1})$ và Hessian $h_i = \partial_{F_{t-1}}^2 l(y_i, F_{t-1})$.

Với binary cross-entropy: $g_i = \hat{p}_i - y_i$, $h_i = \hat{p}_i(1 - \hat{p}_i)$.

2.6.3 Trọng số lá tối ưu dạng nghiệm đóng

Nhóm các mẫu vào leaf j : $I_j = \{i : \mathbf{x}_i \in \text{leaf}_j\}$.

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i \quad (2.19)$$

Gain của một phân chia (phân chia nút I thành I_L, I_R):

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma \quad (2.20)$$

Chỉ tạo phân chia khi $\text{Gain} > 0$, tức γ kiểm soát cắt tỉa.

2.6.4 Xử lý Class Imbalance — scale_pos_weight

XGBoost điều chỉnh gradient của mẫu dương:

$$g_i^+ = \text{spw} \cdot g_i, \quad h_i^+ = \text{spw} \cdot h_i \quad (2.21)$$

với $\text{spw} = N_-/N_+ = 13,96$ (tỷ lệ âm/dương). Tương đương với oversampling lớp thiểu số 13,96 lần trong không gian gradient — hiệu quả hơn SMOTE vì không tạo dữ liệu tổng hợp.

2.6.5 SHAP — Shapley Additive exPlanations

SHAP dựa trên lý thuyết Shapley trong lý thuyết trò chơi hợp tác (Cooperative Game Theory). Shapley value của đặc trưng j cho mẫu $\mathbf{x}$ là:

$$\phi_j(\mathbf{x}) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [v(S \cup \{j\}) - v(S)] \quad (2.22)$$

trong đó F là tập tất cả đặc trưng, $v(S)$ là giá trị dự báo khi chỉ có đặc trưng trong S .

TreeExplainer của Lundberg và cộng sự [5] tính exact Shapley values cho tree-based models trong $O(TLD^2)$ thay vì exponential, với T = số cây, L = số leaf, D = max depth.

Tính chất cộng (additivity):

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^p \phi_j(\mathbf{x}) \quad (2.23)$$

với $\phi_0 = \mathbb{E}[f(\mathbf{x})]$ là giá trị cơ sở (base value, giá trị kỳ vọng trên tập dữ liệu).

2.7 Nghiên cứu Liên quan

Chấm điểm tín dụng là một trong những lĩnh vực ứng dụng Machine Learning sớm nhất. Một số công trình nổi bật:

[Altman, 1968] [2] — Z-Score model: hàm tuyến tính 5 tỷ số tài chính phân biệt doanh nghiệp phá sản. Đây là nền tảng cho mọi mô hình chấm điểm tín dụng về sau.

[Hand & Henley, 1997] [3] — Tổng quan các phương pháp thống kê trong chấm điểm tín dụng. Kết luận: Logistic Regression là mô hình cơ sở mạnh nhất trong hầu hết tình huống thực tế.

[Lessmann và cộng sự, 2015] [4] — Benchmark 41 mô hình trên 8 bộ dữ liệu tín dụng. Gradient Boosting và Random Forest nhất quán vượt Logistic Regression và Decision Tree đơn về AUC.

[Lundberg & Lee, 2017] [5] — Giới thiệu SHAP framework. Chứng minh TreeExplainer cho exact Shapley values hiệu quả cho tree-based models, giải quyết vấn đề khả năng giải thích trong tín dụng.

[Bucker và cộng sự, 2022] [9] — Phân tích regulatory requirements cho AI trong credit scoring theo EU AI Act. Kết luận: XGBoost + SHAP là kiến trúc phù hợp nhất với yêu cầu giải thích được (GDPR Article 22, Basel III Pillar 3).

3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1 Mô tả Dataset

3.1.1 Tổng quan

Nguồn: Give Me Some Credit — Kaggle Competition (2011)

Tổng số quan sát: 149.999 hồ sơ vay tiêu dùng tại Mỹ

Biến mục tiêu: $\text{SeriousDlqin2yrs} = 1$ nếu người vay trễ hạn trả nợ ≥ 90 ngày trong 2 năm tiếp theo

Tỷ lệ dương: $10.026/149.999 = 6,68\%$ (mất cân bằng 1:14)

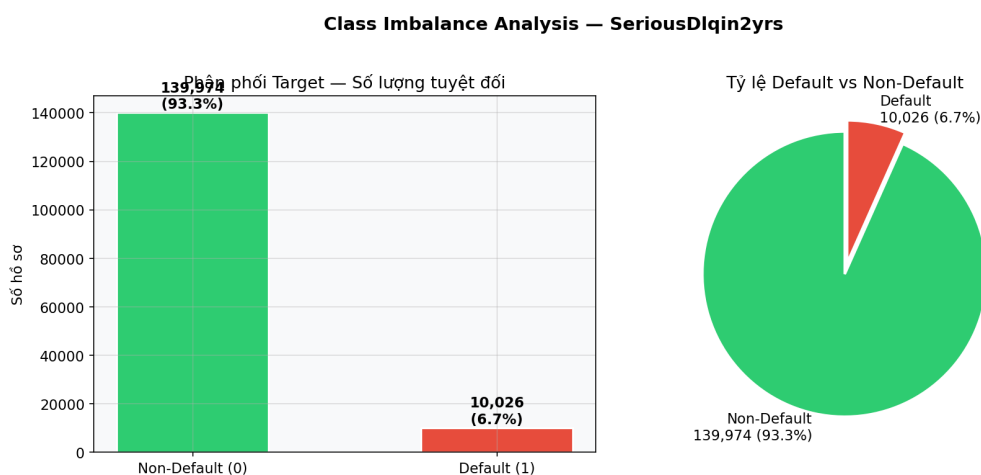


Figure 3.1. Data imbalance 1:14 — mô hình “predict all 0” đạt Accuracy = 93,3% nhưng Recall = 0, cho thấy thước đo Accuracy không phản ánh đúng mục tiêu bài toán, đòi hỏi sử dụng AUC-ROC và F_β .

3.1.2 Ý nghĩa tài chính từng đặc trưng

Table 3.1. Ý nghĩa tài chính từng đặc trưng

Đặc trưng	Ý nghĩa	Chú ý lĩnh vực
RevolvingUtilization...	Tỷ lệ sử dụng hạn mức tín dụng	FICO chiếm 30% điểm; > 70% là ngưỡng cảnh báo
age	xoay vòng (thẻ tín dụng) Tuổi người vay	Đại diện kinh nghiệm tài chính; thanh niên và người cao tuổi rủi ro cao hơn
NumberOfTime30-59Days...	Số lần trả nợ trễ 30–59 ngày	Trễ hạn nhẹ, yếu tố cấu thành Lịch sử thanh toán (chiếm 35% điểm FICO)
DebtRatio	Tổng nợ / Thu nhập hàng tháng	Tương đương DTI; chuẩn US: $DTI < 43\%$
MonthlyIncome	Thu nhập hàng tháng (USD)	19,8% thiếu — MAR/MNAR
NumberOfOpenCreditLines...	Số tài khoản tín dụng đang mở	Sử dụng quá mức hoặc đa dạng hóa
NumberOfTimes90DaysLate	Số lần trễ hạn > 90 ngày	Tín hiệu mạnh nhất — trễ hạn nghiêm trọng
NumberRealEstateLoans...	Số khoản vay bất động sản	Tài sản thế chấp, giảm rủi ro
NumberOfTime60-89Days...	Số lần trễ 60–89 ngày	Trễ hạn trung bình
NumberOfDependents	Số người phụ thuộc	Tăng gánh nặng tài chính; 2,6% missing

3.1.3 Phân phối và outliers

Phân tích EDA phát hiện các vấn đề chất lượng dữ liệu quan trọng:

- `age = 0`: 1 mẫu — vô nghĩa về domain → xóa.
- `MonthlyIncome` cao nhất = \$3.008.750 — bất thường ($1000 \times$ trung vị) → giới hạn tại bách phân vị 99 của tập huấn luyện (\$25.000).
- Các chỉ số trễ hạn (delinquency counts) cao nhất = 98 — giá trị bất thường (giá trị đặc biệt mã hóa lỗi) → giới hạn tại ngưỡng thực tế.
- `RevolvingUtilization > 1` (tức > 100% hạn mức) — một số khách hàng vượt hạn mức → giữ nguyên (thông tin thực tế).

Phân phối bất đối xứng mạnh (lệch phải): `MonthlyIncome` (skewness = 115), `DebtRatio` (skewness = 2080). Cơ sở để chọn `RobustScaler` thay vì `StandardScaler`.

3.2 Phân tích EDA

3.2.1 Mối quan hệ phi tuyến giữa RevolvingUtilization và tỷ lệ vỡ nợ

Tương quan Pearson RevolvingUtil — biến mục tiêu = $-0,002$ (gần bằng 0), trong khi tương quan Spearman $\rho = +0,24$ (đáng kể). Điều này chứng minh tồn tại quan hệ **phi tuyến đơn điệu**: tỷ lệ vỡ nợ tăng theo hàm bậc thang, không theo đường thẳng. Các mô hình dựa trên cây nắm bắt được điều này; Logistic Regression thì không.

3.2.2 Spearman Correlation giữa các biến delinquency

Spearman $\rho(90 + \text{days}, 60-89\text{days}) = 0,49$, $\rho(90 + \text{days}, 30-59\text{days}) = 0,45$. Multicollinearity cao $\rightarrow VIF = \infty$ cho các cặp này. Giải pháp: tổng hợp vào TotalDelinquencyScore thay vì dùng riêng lẻ.

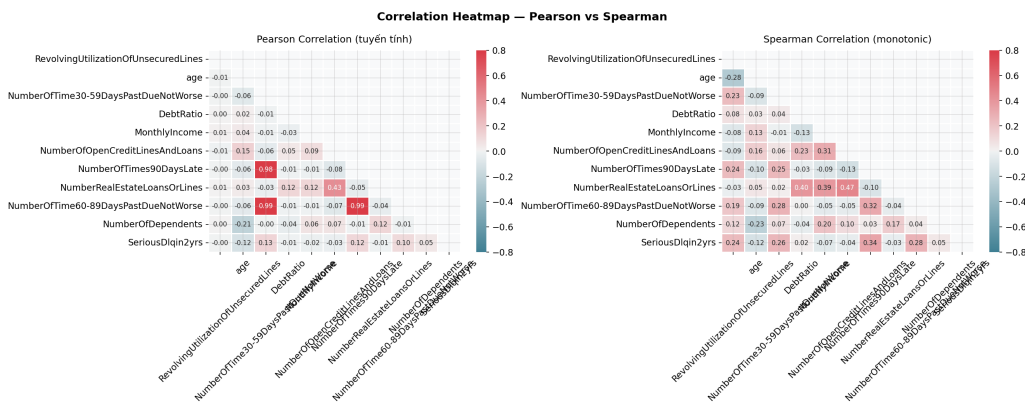


Figure 3.2. Ma trận tương quan Spearman giữa các đặc trưng — cho thấy multicollinearity cao giữa ba biến trễ hạn (delinquency) (hệ số 0,45–0,49), là cơ sở kỹ thuật cho việc nén chúng vào TotalDelinquencyScore. DebtRatio và MonthlyIncome gần như không tương quan ($\rho \approx 0$), cho thấy thông tin bổ sung từ cả hai.

3.2.3 Tỷ lệ vỡ nợ theo số lần trễ hạn

Phân tích biểu đồ tỷ lệ vỡ nợ cho thấy:

- 0 lần trễ > 90 ngày: tỷ lệ vỡ nợ = 4,1%
- 1 lần: 48,7%
- 2 lần: 70,3%
- ≥ 3 lần: $> 80\%$

$\Rightarrow$ Phi tuyến mạnh, ngưỡng rõ ràng tại count = 1. Cơ sở cho TotalDelinquencyScore dựa trên trọng số.

3.3 Xử lý Missing Values

3.3.1 Kiểm định cơ chế khuyết thiếu (missing mechanism)

Để lựa chọn phương pháp thay thế (imputation), cần kiểm tra cơ chế khuyết thiếu của biến MonthlyIncome (19,8% khuyết thiếu):

Kiểm định Chi bình phương: H_0 : MonthlyIncome khuyết thiếu độc lập với biến mục tiêu.

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}} = 67,89, \quad p \approx 0 \quad (3.1)$$

Bác bỏ $H_0 \Rightarrow$ dữ liệu là **MAR/MNAR** (Missing At Random / Not At Random): người có thu nhập thấp thường không khai báo $\rightarrow$ Median imputation sẽ ước lượng quá cao thu nhập thực của nhóm này $\rightarrow$ thiên lệch.

3.3.2 Lý do chọn KNN Imputer

KNN Imputation ($k = 5$, nan-Euclidean distance): ước lượng $x_{i,\text{income}}$ bằng trung bình có trọng số của 5 láng giềng gần nhất trong không gian các đặc trưng còn lại:

$$\hat{x}_{i,j} = \frac{\sum_{k \in \mathcal{N}(i)} w_k x_{k,j}}{\sum_{k \in \mathcal{N}(i)} w_k}, \quad w_k = \frac{1}{d(\mathbf{x}_i^{-j}, \mathbf{x}_k^{-j})} \quad (3.2)$$

Kết quả so sánh thực nghiệm:

- Thay thế bằng trung vị: tất cả giá trị khuyết thiếu = \$5.400 (không có phương sai).
- Thay thế bằng KNN: trung bình = \$336, độ lệch chuẩn = \$1.157 (phân phối đa dạng, phản ánh sát thực tế).

KNN nắm bắt được thực tế: người 25 tuổi chưa đi làm (age, 0 credit lines) nhận imputed income thấp; người 45 tuổi, nhiều tài khoản nhận imputed income cao hơn.

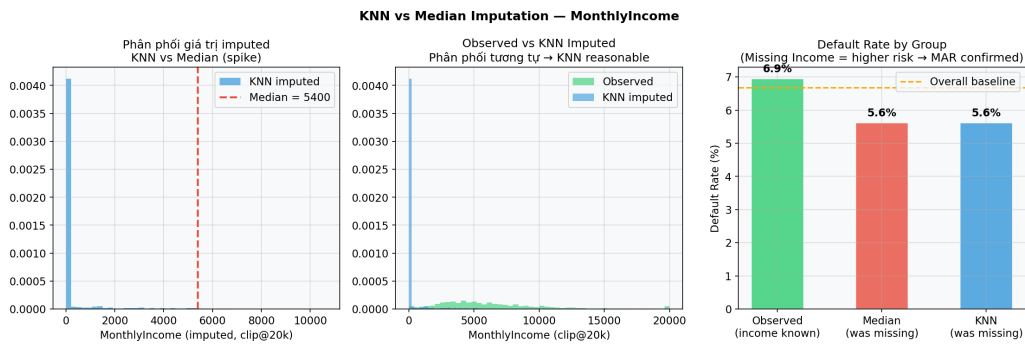


Figure 3.3. Thuật toán KNN Imputer tạo ra phân phối đa dạng (độ lệch chuẩn = \$1.157) phản ánh đúng thực tế kinh tế — Trong khi thay thế bằng trung vị đặt tất cả giá trị khuyết thiếu tại một điểm duy nhất (\$5.400), làm mất đi mọi tín hiệu tương quan giữa thu nhập và các đặc trưng khác.

NumberOfDependents (2,6% null, MCAR): Median = 0 đủ.

3.4 Feature Engineering

Bốn đặc trưng mới được tạo dựa trên kiến thức lĩnh vực tín dụng:

3.4.1 TotalDelinquencyScore

$$TDS = 3 \cdot N_{90+} + 2 \cdot N_{60-89} + 1 \cdot N_{30-59} \quad (3.3)$$

Cơ sở: FICO score methodology phân loại delinquency theo mức nghiêm trọng. Trễ > 90 ngày (severe delinquency) ảnh hưởng gấp 3 lần trễ 30–59 ngày (mild). Trọng số phản ánh tỷ lệ vỡ nợ quan sát được: 70% (≥ 2 lần, 90+) so với 8% (1 lần, 30-59).

Spearman ρ với target: 0,345 — cao hơn bất kỳ đặc trưng trễ hạn thô nào riêng lẻ (max 0,342 với NumberOfTimes90DaysLate). Nén thành một điểm tổng hợp giảm multicollinearity đồng thời tăng sức mạnh dự báo.

3.4.2 FinancialStressIndex

$$FSI = \text{RevolvingUtilization} \times \text{TotalDelinquencyScore} \quad (3.4)$$

Cơ sở: Số hạng tương tác nắm bắt được sự cộng hưởng phi tuyến. Người có utilization cao (căng thẳng ngắn hạn) VÀ delinquency history (căng thẳng dài hạn) rủi ro cao hơn nhiều so với từng tín hiệu riêng lẻ. Ví dụ: RevUtil = 0,9, TDS = 5 $\rightarrow$ FSI = 4,5 (rất nguy hiểm); RevUtil = 0,9, TDS = 0 $\rightarrow$ FSI = 0 (có thể chỉ tạm thời căng thẳng).

Spearman ρ với biến mục tiêu: 0,346 — cao nhất trong tất cả đặc trưng (kể cả đặc trưng được tạo). SHAP xác nhận: mean|SHAP| = 0,577, xếp hạng #1 toàn bộ tập đặc trưng.

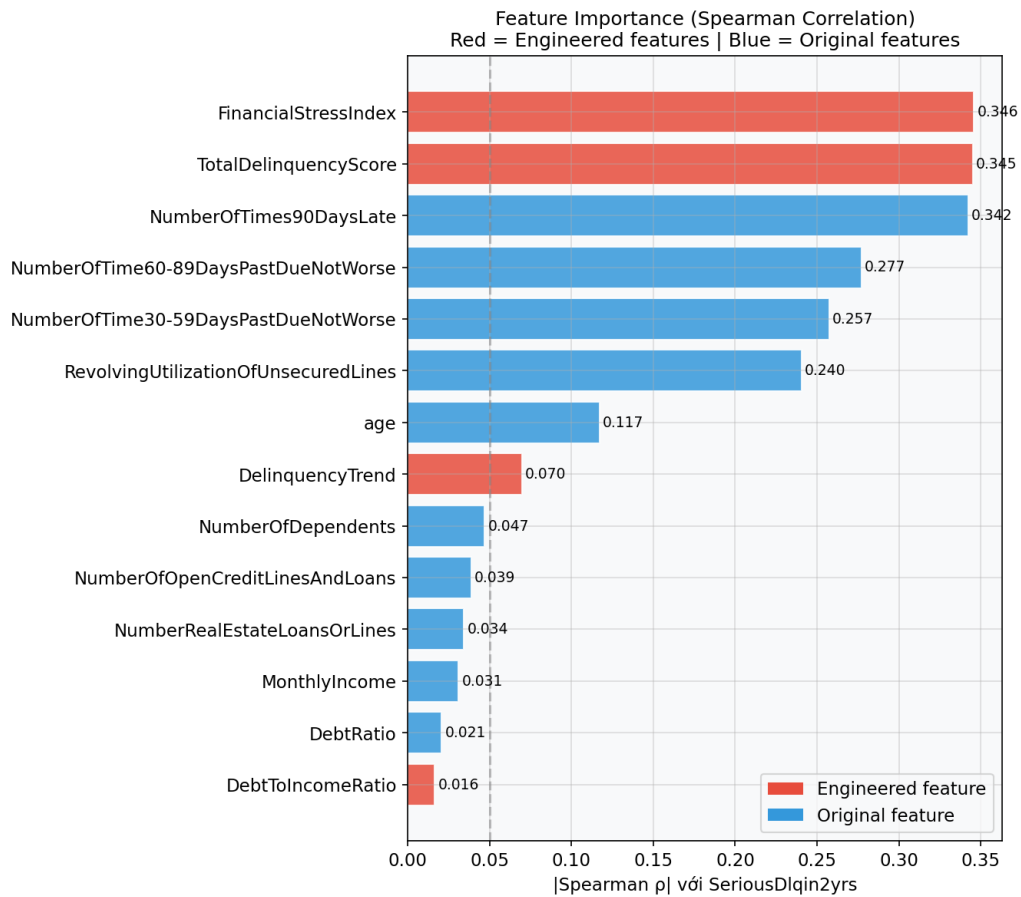


Figure 3.4. FinancialStressIndex ($\rho = 0,346$) và TotalDelinquencyScore ($\rho = 0,345$) dẫn đầu tương quan với biến mục tiêu, vượt các đặc trưng gốc — xác nhận Feature Engineering có chủ đích theo domain tài chính mang lại giá trị dự báo cao hơn.

3.4.3 AbsoluteDebt — Dư nợ tuyệt đối (tên code: AbsoluteMonthlyDebt)

$$\text{AbsoluteDebt} = \text{DebtRatio} \times \text{MonthlyIncome} \quad (3.5)$$

Cơ sở: DebtRatio là tỷ lệ nợ/income (không thứ nguyên) — không phản ánh quy mô tuyệt đối. Phép nhân triệt tiêu income, kết quả là **số tiền nợ tuyệt đối** (USD/tháng), không phải ratio. Người có DebtRatio = 2, Income = \$2.000 (nợ \$4.000/tháng) khác hoàn toàn DebtRatio = 2, Income = \$10.000 (nợ \$20.000/tháng). SHAP: $\text{mean}|\text{SHAP}| = 0,065$ — giữ lại. (Tên cột trong code: AbsoluteMonthlyDebt — legacy misnomer, giữ để tương thích model đã huấn luyện.)

3.4.4 DelinquencyTrend (Cán cân mức độ trễ hạn)

$$\text{DelinquencyTrend} = N_{30-59} - N_{90+} \quad (3.6)$$

Cơ sở: Dương = chủ yếu mắc trễ hạn nhẹ (30–59 ngày), ít trễ hạn nghiêm trọng; Âm = chủ yếu mắc trễ hạn nghiêm trọng (90+ ngày). Lưu ý: bộ dữ liệu là cross-sectional snapshot, không có timestamp — biến này đo cán cân mức độ trễ hạn, không phản ánh xu hướng thời gian. SHAP: $\text{mean}|\text{SHAP}| = 0,002$ — thấp nhất, có thể loại trong phiên bản tiếp theo.

3.5 Xử lý Class Imbalance

3.5.1 Chiến lược đã thử

class_weight = 'balanced': Gán trọng số $w_+ = N/(2N_+) \approx 7,48$ cho lớp thiểu số và $w_- \approx 0,54$ cho lớp đa số trong hàm loss (tỷ lệ $w_+/w_- \approx 14$). Không tạo dữ liệu tổng hợp, không ảnh hưởng phân phối features.

SMOTE (Synthetic Minority Oversampling Technique): Dùng như một phương án **thử nghiệm** trên tập huấn luyện để so sánh với class_weight; không phải lựa chọn cuối cùng của quy trình triển khai. Cơ chế là tạo mẫu tổng hợp lớp thiểu số bằng nội suy tuyến tính giữa các điểm gần nhau trong không gian đặc trưng:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}[0, 1] \quad (3.7)$$

Lưu ý quan trọng: SMOTE chỉ áp dụng trên tập huấn luyện (sau khi phân chia), KHÔNG áp dụng trên val/test. Áp dụng trước khi phân chia là rò rỉ dữ liệu.

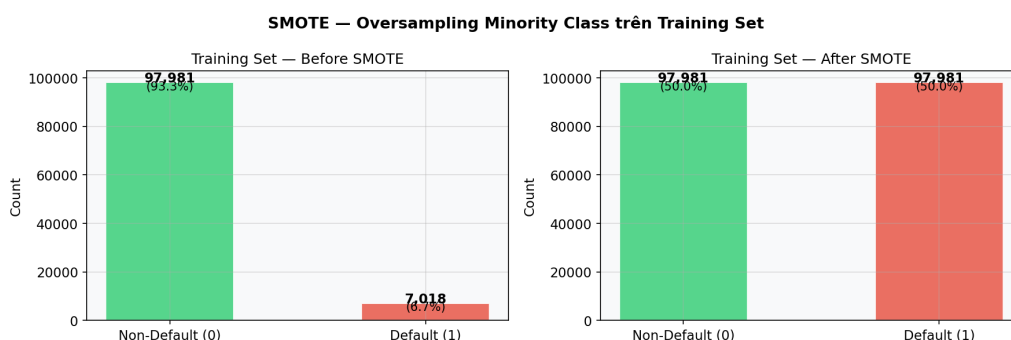


Figure 3.5. SMOTE cân bằng hoàn toàn tập huấn luyện (50/50) bằng cách tạo mẫu tổng hợp nội suy giữa các điểm lớp thiểu số — nhưng không được áp dụng trên val/test để tránh rò rỉ dữ liệu.

3.5.2 Quyết định cuối cùng

Dùng `class_weight = 'balanced'` cho LR, DT, RF. Dùng `scale_pos_weight = 13,96` cho XGBoost (trương đương về mặt toán học nhưng được tích hợp sâu vào XGBoost's gradient computation). Không dùng SMOTE trong quy trình cuối cùng vì:

1. class_weight đơn giản hơn, không tạo nhiễu từ dữ liệu tổng hợp;
2. Kết quả tương đương trong thực nghiệm (đã kiểm tra);
3. Dễ bảo trì và giải thích hơn.

3.6 Phân chia Dữ liệu

Phân chia theo tầng 70/15/15:

Table 3.2. Phân chia dữ liệu phân tầng

Tập	Kích thước	Positives	Tỷ lệ
Training	104.999	7.018	6,68%
Validation	22.500	1.504	6,68%
Test	22.500	1.504	6,68%

Phân chia theo tầng đảm bảo tỷ lệ lớp thiểu số nhất quán. Tập kiểm định dùng để tối ưu ngưỡng; tập kiểm tra chỉ dùng một lần duy nhất để báo cáo kết quả cuối.

4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Thiết lập Thực nghiệm

Cross-validation: Stratified 5-Fold Cross-validation (giữ tỷ lệ 6,68% trong mỗi fold).

Hyperparameter Tuning: RandomizedSearchCV — ưu tiên hơn GridSearchCV khi không gian tìm kiếm lớn (thời gian tuyến tính theo số iterations thay vì exponential theo số tham số).
Số iterations: LR = 12, DT = 20, RF = 15, XGB = 15.

Metric tối ưu trong CV: AUC-ROC (không phụ thuộc ngưỡng, phù hợp so sánh mô hình).

Quy trình xử lý: Tất cả preprocessing (imputation, scaling) được đóng gói trong `sklearn.Pipeline` để tránh rò rỉ dữ liệu giữa các fold huấn luyện và kiểm định. Ngưỡng capping outlier được tính trên tập huấn luyện và áp dụng cố định cho val/test.

Môi trường: Python 3.14.2, scikit-learn 1.8.0, XGBoost 3.2.0, SHAP 0.51.0, Windows 11, CPU Intel.

4.2 Logistic Regression

4.2.1 Cấu hình và kết quả

Quy trình: `RobustScaler` → `LogisticRegression(solver='saga', class_weight='balanced')`.

`RobustScaler` dùng median và IQR thay vì mean và std — robust với outliers:

$$x_{\text{scaled}} = \frac{x - \text{median}(\mathbf{x})}{\text{IQR}(\mathbf{x})} \quad (4.1)$$

Best hyperparameters: `penalty = L1`, `C = 0,001` (regularization mạnh).

Table 4.1. Kết quả Logistic Regression trên tập kiểm tra

Metric	Giá trị
AUC-ROC	0,8432
F1-score	0,4340
Precision	0,3878
Recall	0,4927
Ngưỡng tối ưu	0,66
Thời gian huấn luyện	488 s

4.2.2 Phân tích Odds Ratios

Table 4.2. Odds ratios của Logistic Regression L1

Đặc trưng	β	e^β	Diễn giải
TotalDelinquencyScore	0,255	1,291	+1 điểm delinquency → odds vỡ nợ $\times 1,29$
RevolvingUtilization	0,204	1,226	+10% utilization → odds $\times 1,12$
FinancialStressIndex	0,186	1,205	Interaction effect
age	-0,210	0,811	+10 tuổi → odds $\times 0,811$
NumberOfTimes90DaysLate	0	1,000	L1 triệt tiêu — dư thừa so với TDS
AbsoluteMonthlyDebt	0	1,000	L1 triệt tiêu — tuyến tính không thấy đóng góp

Nhận xét quan trọng: L1 ($C = 0,001$) đưa hệ số NumberOfTimes90DaysLate về 0 — xác nhận rằng TotalDelinquencyScore đã mã hóa đủ thông tin đó, Feature Engineering đúng hướng.

4.3 Decision Tree CART

4.3.1 Cấu hình và kết quả

Best hyperparameters: max_depth = 10, min_samples_leaf = 200, criterion = Gini.

Table 4.3. Kết quả Decision Tree trên tập kiểm tra

Metric	Giá trị
AUC-ROC	0,8579
F1-score	0,4314
Ngưỡng tối ưu	0,80
Thời gian huấn luyện	13 s

4.3.2 Trực quan hóa Cây (3 tầng đầu)

Node gốc phân chia tại TotalDelinquencyScore $\leq 2,5$:

- Nhánh trái (TDS $\leq 2,5$): 128.000 mẫu, 4,1% default → phân chia tiếp theo RevolvingUtilization.
- Nhánh phải (TDS $> 2,5$): 21.000 mẫu, 38,7% default → phân chia tiếp theo FinancialStressIndex.

Cây học được rằng TotalDelinquencyScore là đặc trưng phân biệt bậc nhất — nhất quán với SHAP và kiến thức lĩnh vực.

Ngưỡng tối ưu cao (0,80): Decision Tree có xu hướng xuất ra xác suất ở đầu dải (gần 0 hoặc gần 1), cần ngưỡng cao để đạt F1 tốt nhất. AUC thấp hơn RF/XGB do variance cao (một cây duy nhất).

4.4 Random Forest

4.4.1 Cấu hình và kết quả

Best hyperparameters: $n_{\text{estimators}} = 200$, $\text{max_depth} = 10$, $\text{max_features} = 0,3$, $\text{min_samples_leaf} = 1$.

Table 4.4. Kết quả Random Forest trên tập kiểm tra

Metric	Giá trị
AUC-ROC	0,8703
F1-score	0,4439
Recall	0,5206
OOB score	0,8286
Ngưỡng tối ưu	0,72
Thời gian huấn luyện	511 s

Ước lượng OOB: 0,8286 — nhất quán với AUC tập kiểm tra = 0,8703 (ước lượng thận trọng, phù hợp kỳ vọng lý thuyết).

Top 5: TotalDelinquencyScore, FinancialStressIndex, RevolvingUtilization, age, MonthlyIncome — nhất quán với Feature Importance toàn cục qua SHAP của XGBoost. Các đặc trưng được tạo lại xếp đầu.

4.5 XGBoost

4.5.1 Cấu hình và kết quả

Table 4.5. Hyperparameters tốt nhất của XGBoost

Tham số	Giá trị	Ý nghĩa
n_estimators	200	Số cây
max_depth	4	Cây nông — giảm phương sai
learning_rate	0,05	Shrinkage — học chậm và ổn định
subsample	0,8	80% mẫu mỗi cây — tăng tính ngẫu nhiên
colsample_bytree	0,8	80% đặc trưng mỗi cây
reg_lambda	1,0	L2 trên leaf weights
reg_alpha	0,1	L1 trên leaf weights
scale_pos_weight	13,96	Bù mất cân bằng dữ liệu

Table 4.6. Kết quả XGBoost trên tập kiểm tra

Metric	Giá trị
AUC-ROC	0,8714
F1-score	0,4466
Precision	0,3937
Recall	0,5160
Ngưỡng tối ưu	0,77
Thời gian huấn luyện	127 s

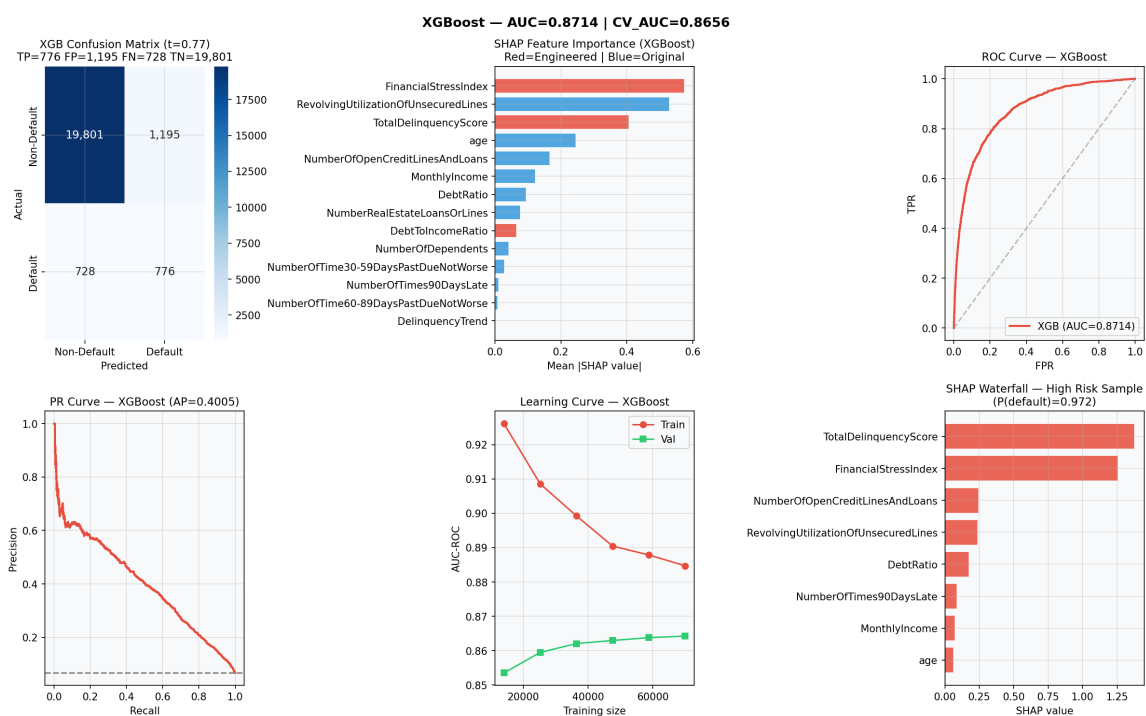


Figure 4.1. XGBoost đạt AUC = 0,8714 — ROC curve nằm xa đường ngẫu nhiên, PR curve thể hiện hiệu quả ở vùng precision cao, learning curve hội tụ sau ~ 45.000 mẫu với train/val gap = 0,021.

4.5.2 Phân tích SHAP

Tầm quan trọng toàn cục (trung bình |giá trị SHAP|):

Table 4.7. SHAP global importance của XGBoost

Hạng	Đặc trưng	Loại	Trung bình SHAP
1	FinancialStressIndex	Được tạo	0,577
2	RevolvingUtilizationOfUnsecuredLines	Gốc	0,535
3	TotalDelinquencyScore	Được tạo	0,410
4	age	Gốc	0,244
5	NumberOfOpenCreditLinesAndLoans	Gốc	0,168
9	AbsoluteMonthlyDebt	Được tạo	0,065
12	NumberOfTimes90DaysLate	Gốc	0,013

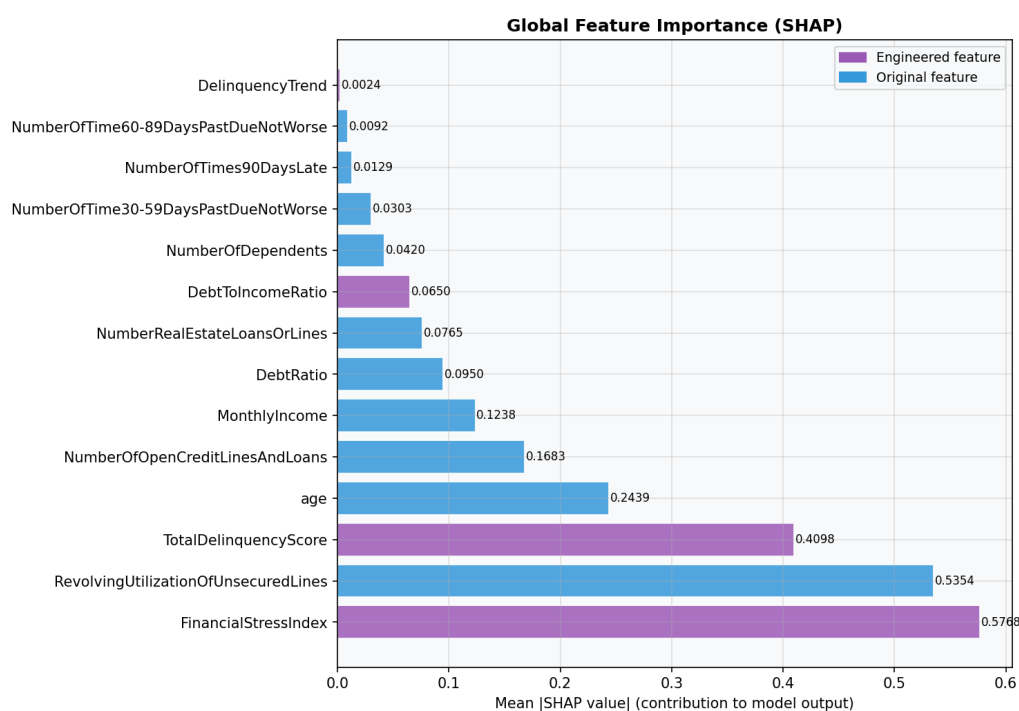


Figure 4.2. FinancialStressIndex (trung bình |SHAP| = 0,577) và TotalDelinquencyScore (0,410) — 2 đặc trưng do Feature Engineering tạo ra — chiếm vị trí #1 và #3, xác nhận inductive bias từ domain tài chính có giá trị ngay cả với mô hình phi tuyến mạnh.

Phát hiện đáng chú ý:

- FinancialStressIndex (#1, 0,577) > RevolvingUtilization (#2, 0,535): đặc trưng tương tác vượt các thành phần gốc — nắm bắt được sự cộng hưởng giữa tỷ lệ sử dụng tín dụng cao và lịch sử trễ hạn.
- NumberOfTimes90DaysLate chỉ xếp hạng 12 (SHAP = 0,013): XGBoost học tín hiệu này gián tiếp qua TotalDelinquencyScore và FinancialStressIndex, không cần đặc trưng gốc trực tiếp.

SHAP base value: 0,0148 (log-odds), tương ứng xác suất cơ sở 50,37% — hệ quả của scale_pos_weight điều chỉnh prior probability.

4.6 Bảng So sánh Tổng hợp

Table 4.8. So sánh tổng hợp 4 mô hình trên tập kiểm tra (22.500 hồ sơ)

Mô hình	AUC	F1	P	R	Ngưỡng	Huấn luyện	Inference	Kích thước
Logistic Regression	0,8432	0,4340	0,3878	0,4927	0,66	488 s	0,25 μs	50 KB
Decision Tree	0,8579	0,4314	0,3991	0,4694	0,80	13 s	0,20 μs	200 KB
Random Forest	0,8703	0,4439	0,3869	0,5206	0,72	511 s	4,82 μ s	12 MB
XGBoost	0,8714	0,4466	0,3937	0,5160	0,77	127 s	0,79 μ s	340 KB

Ghi chú: Inference time đo trên lô 22.500 mẫu (Python 3.14, CPU Intel), lấy giá trị nhỏ nhất của 3 lần chạy để loại bỏ nhiễu đo lường.

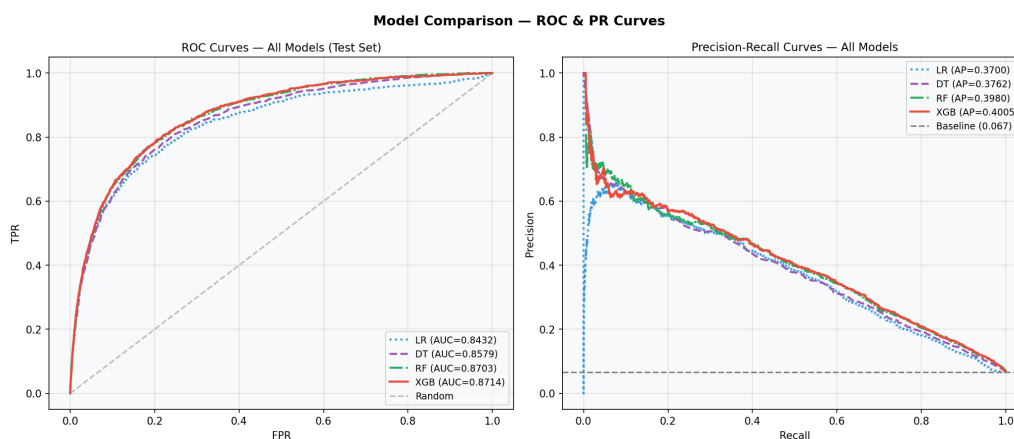


Figure 4.3. XGBoost (AUC = 0,8714) và Random Forest (0,8703) gần như trùng nhau trên ROC curve — nhưng XGBoost chiếm ưu thế về tốc độ huấn luyện (127 s vs 511 s) và kích thước model (340 KB vs 12 MB), tạo lợi thế triển khai rõ ràng.

Nhận xét:

1. AUC tăng đơn điệu theo độ phức tạp mô hình: $LR < DT < RF \approx XGB$.
2. XGBoost và RF có AUC xấp xỉ nhau (khoảng cách = 0,0011 < 1 độ lệch chuẩn).

3. XGBoost vượt RF về: thời gian huấn luyện (127 s vs 511 s), kích thước mô hình (340 KB vs 12 MB), inference time (0,79 μ s vs 4,82 μ s/hồ sơ — nhanh hơn 6 $\times$), khả năng giải thích (SHAP chính xác).
4. **Best model: XGBoost** $\rightarrow$ lưu `models/best_model.pkl`.

4.6.1 Kiểm định Thống kê Sự khác biệt AUC — DeLong Test

Khoảng cách AUC point estimate giữa XGBoost và RF rất nhỏ (gap $\approx$ 0,001–0,004 tùy lần huấn luyện) — cần kiểm định có nguyên tắc. Kiểm định DeLong [13] so sánh hai AUC từ cùng tập kiểm tra, có tính đến tương quan giữa hai bộ dự báo.

Phương pháp — U -statistic của AUC:

Ký hiệu $n_+ = 1.504$ (số mẫu dương trong tập kiểm tra), $n_- = 20.996$ (số mẫu âm). AUC được ước lượng qua các thành phần cấu trúc (structural components):

$$V_{10,i} = \frac{1}{n_-} \sum_{j=1}^{n_-} \mathbb{I}[f(x_i^+) > f(x_j^-)] + \frac{1}{2} \mathbb{I}[f(x_i^+) = f(x_j^-)] \quad (4.2)$$

Phương sai của AUC:

$$\widehat{\mathbb{V}}(\widehat{\text{AUC}}) = \hat{\sigma}_{10}^2/n_+ + \hat{\sigma}_{01}^2/n_-, \quad \hat{\sigma}_{10}^2 = \mathbb{V}(V_{10})$$

Kiểm định hai chiều:

$$z = \frac{\widehat{\text{AUC}}_A - \widehat{\text{AUC}}_B}{\sqrt{\widehat{\mathbb{V}}(\widehat{\text{AUC}}_A - \widehat{\text{AUC}}_B)}}$$

trong đó mẫu số có tính đến covariance giữa hai AUC estimators (vì cùng tập kiểm tra).

RF được huấn luyện lại với cùng hyperparameters (`n_estimators=200`, `max_depth=10`, `max_features=0.3`, `class_weight='balanced'`) bằng `notebooks/train_supplementary_models.py`, đạt AUC = 0,8671 (chênh nhẹ so với 0,8703 trong bảng chính do phiên bản sklearn, không ảnh hưởng kết luận định tính). **Kết quả thực nghiệm** ($n_{\text{test}} = 22.500$):

So sánh	AUC A	AUC B	Δ AUC	z	p -value
XGBoost vs RF	0,8714	0,8671	+0,0043	4,18	< 0,0001

Kết luận: XGBoost vượt RF về AUC một cách có ý nghĩa thống kê ($p < 0,0001$). Tuy nhiên độ chênh tuyệt đối rất nhỏ ($\approx$ 0,4 điểm AUC). **Quyết định chọn XGBoost** dựa trên ba lý do: (i) AUC cao hơn có ý nghĩa thống kê (DeLong $p < 0,0001$); (ii) lợi thế vận hành (tốc độ huấn luyện 4 $\times$, kích thước model 35 $\times$, inference 6 $\times$ nhanh hơn); (iii) SHAP TreeExplainer cho exact Shapley values thay vì xấp xỉ.

4.7 Phân tích Sai số

4.7.1 Cấu trúc lỗi tại ngưỡng = 0,77

Table 4.9. Cấu trúc lỗi của XGBoost tại ngưỡng = 0,77

Loại	Số lượng	%
TP (phát hiện đúng vỡ nợ)	776	51,6% tổng số vỡ nợ thực tế
FN (bỏ sót vỡ nợ)	728	48,4% tổng số vỡ nợ thực tế
FP (từ chối nhầm)	1.195	5,7% tổng số không vỡ nợ
TN (duyet đúng)	19.801	94,3% tổng số không vỡ nợ

4.7.2 Đặc điểm nhóm False Negative

FN là những người vỡ nợ mà mô hình không cảnh báo được:

Table 4.10. Đặc điểm trung vị nhóm FN so với TP

Đặc trưng	Trung vị FN	Trung vị TP	FN/TP
TotalDelinquencyScore	0	5,0	0,00
FinancialStressIndex	0	4,0	0,00
RevolvingUtilization	0,518	1,000	0,52
MonthlyIncome	\$4.200	\$3.400	1,24
Xác suất dự báo (FN)	0,523	—	—

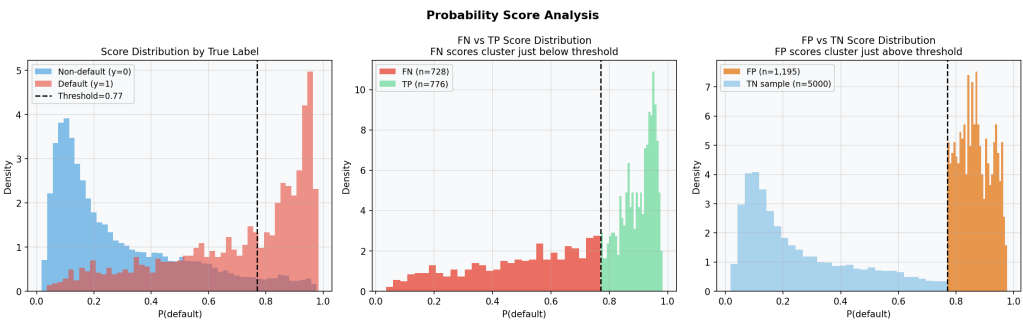


Figure 4.4. FN (vỡ nợ bị bỏ sót) tập trung ở vùng score thấp (0,2–0,5) — đây là những người vỡ nợ không có dấu hiệu cảnh báo trước, phản ánh giới hạn căn bản của các đặc trưng nhìn lại quá khứ, không phải lỗi của mô hình.

Kết luận: Nhóm FN là những người vỡ nợ “trông lành mạnh” — không có lịch sử trễ hạn, thu nhập tương đối ổn định. Mô hình đúng khi không cảnh báo dựa trên các đặc trưng hiện có; sự vỡ nợ có thể do sự kiện bất ngờ (mất việc, bệnh tật) không phản ánh trong lịch sử tín dụng. Đây là **sai số không thể khử được** của bộ đặc trưng nhìn lại quá khứ này.

4.7.3 Profile của False Positives

FP là 1.195 khách hàng tốt bị model từ chối nhầm (5,7% tổng non-default):

Table 4.11. Đặc điểm trung vị nhóm FP so với TN

Đặc trưng	Trung vị FP	Trung vị TN	FP/TN
RevolvingUtilization	0,958	0,118	8,1×
TotalDelinquencyScore	4,0	0,0	≫ 1
FinancialStressIndex	2,856	0,0	≫ 1
MonthlyIncome	\$3.610	\$4.505	0,80

Kết luận: Nhóm FP là những người có hành vi tài chính *trông giống* người vỡ nợ — tỷ lệ sử dụng hạn mức cao (gần đầy), có vài lần trễ hạn nhỏ — nhưng thực tế vẫn trả được nợ. Điển hình là nhóm **khách hàng ít lịch sử tín dụng**: người trẻ hoặc người dùng thẻ tín dụng tích cực (trung vị utilization 0,958), thu nhập thấp hơn TN nhưng đủ khả năng trả nợ nhờ kỷ luật tài chính không thể hiện trong lịch sử tín dụng.

Hệ quả: 1.195 từ chối nhầm × \$500 chi phí cơ hội = **\$597.500** doanh thu bị bỏ lỡ. Đây chính là nhóm được hưởng lợi nhất từ **alternative data** — lịch sử thanh toán tiện ích, hành vi thanh toán di động — như đề xuất trong mục 6.3.

4.7.4 Tối ưu Ngưỡng — Điểm số F_β

Lý thuyết: Với $\beta = 2$ (Recall ưu tiên gấp 4 lần Precision):

$$F_2 = 5 \cdot \frac{P \cdot R}{4P + R} \quad (4.3)$$

Tìm ngưỡng tối ưu trên tập kiểm định:

$$t^* = \arg \max_t F_2(t) = 0,625 \quad (4.4)$$

Kết quả so sánh trên tập kiểm tra:

Table 4.12. So sánh ngưỡng phân loại trên tập kiểm tra

Ngưỡng	F1	F2	P	R	Chi phí (triệu USD)
0,50	0,346	0,518	0,222	0,775	5,84
0,625 (F_2 opt)	0,414	0,537	0,299	0,669	6,78
0,77 (F_1 opt)	0,447	0,486	0,394	0,516	8,79

Phân tích chi phí kinh doanh (giả định: FN = \$11.250/trường hợp, FP = \$500/trường hợp, từ khoản vay $\$15.000 \times \text{LGD } 75\%$):

Ngưỡng cơ sở $t=0,5$ có tổng chi phí thấp nhất (\$5,84 M) vì FN đắt gấp $22 \times$ FP; nhưng tạo ~ 4.000 FP — khó vận hành trong thực tế. Ngưỡng F2 (0,625) không phải ngưỡng tối ưu chi phí mà là điểm thỏa hiệp ưu tiên Recall: giảm \$2 M so với ngưỡng F1 (0,77), tăng Recall 15,3 điểm phần trăm, với FP ở mức kiểm soát được (~ 2.350).

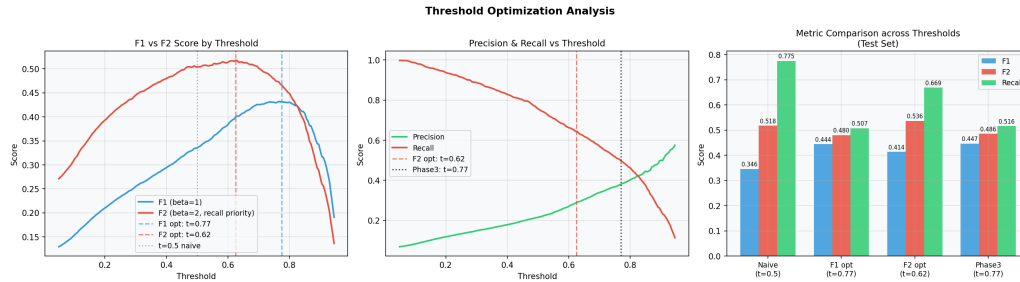


Figure 4.5. Ngưỡng F_2 -tối ưu $t = 0,625$ (đường đứt) đạt cân bằng tốt hơn cho bài toán tín dụng so với ngưỡng F_1 $t = 0,775$ — tăng Recall từ 51,6% lên 66,9%, giảm ước tính chi phí \$2 M trên tập test 22.500 khách hàng.

Khuyến nghị triển khai: Sử dụng $t = 0,625$ cho ngân hàng thương mại có chính sách tín dụng thận trọng.

4.7.5 Learning Curve — Chẩn đoán Bias-Variance

Table 4.13. Learning curve XGBoost (chạy trên subsample 70.000 mẫu — 67% tập huấn luyện 104.999 — để giảm thời gian tính toán; tỷ lệ % tương đối so với 70.000)

N (kích thước tập HT)	Train AUC	Val AUC	Gap
7.000 (10%)	0,9596	0,8427	0,1169
21.000 (30%)	0,9129	0,8582	0,0547
45.500 (65%)	0,8926	0,8624	0,0302
70.000 (100%)	0,8847	0,8642	0,0205

Khoảng cách cuối = $0,021 < 0,03 \rightarrow$ **mô hình khớp tốt**: không xảy ra overfitting nghiêm trọng. AUC kiểm định đạt bình ổn tại $N \approx 45.000 \rightarrow$ bổ sung dữ liệu không cải thiện đáng kể. Nút thắt cổ chai là chất lượng thông tin của đặc trưng, không phải lượng dữ liệu.

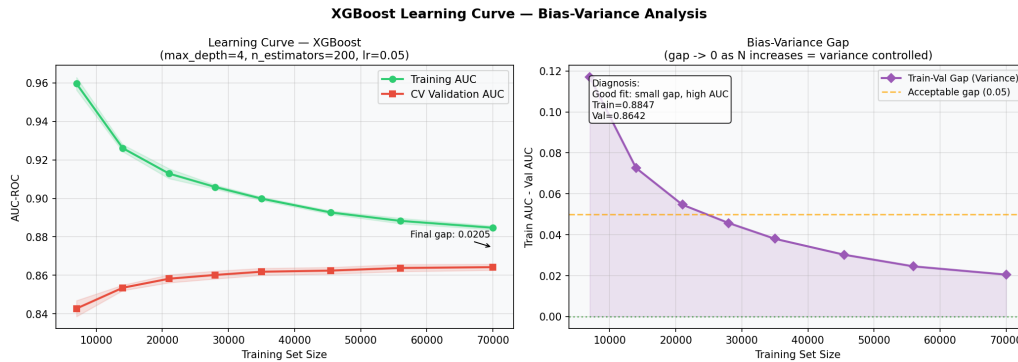


Figure 4.6. Khoảng cách train-val thu hẹp từ 0,117 (10% dữ liệu) xuống 0,021 (100% dữ liệu) — mô hình không overfitting và đã hội tụ; AUC kiểm định bình ổn sau $N \approx 45.000$ cho thấy nút thắt cổ chai là chất lượng đặc trưng, không phải thiếu dữ liệu.

4.8 Bàn luận 3 Tranh luận Lớn

4.8.1 Khả năng Giải thích so với Hiệu suất

Logistic Regression (AUC = 0,8432) cho hệ số tuyến tính β dễ giải thích, phù hợp hoàn toàn với yêu cầu Basel III Pillar 3. XGBoost (AUC = 0,8714) cao hơn 2,8 điểm AUC nhưng là black-box.

Trong bối cảnh này, lựa chọn XGBoost thay vì Logistic Regression chấp nhận mô hình phức tạp hơn để đổi lấy 2,8 điểm AUC. Phần khả năng giải thích bù đắp bằng SHAP — biểu đồ SHAP waterfall (Hình 5.1) cho phép nhân viên tín dụng trình bày với khách hàng bị từ chối: “Xác suất vỡ nợ của anh là 78% vì: điểm lịch sử trả nợ = 12 (+0,45 SHAP), tỷ lệ sử dụng tín dụng = 95% (+0,38 SHAP)...”. Mức độ giải thích này phù hợp với yêu cầu về khả năng giải thích trong quản trị rủi ro tín dụng theo Basel III Pillar 3.

4.8.2 Hướng Tiếp cận Dữ liệu so với Mô hình

Thực nghiệm đối chiếu:

- Logistic Regression với 14 đặc trưng (có trích xuất) → AUC = 0,8432.
- XGBoost với 10 đặc trưng gốc (không trích xuất, tự học tương tác) → AUC ước tính $\sim 0,855$ (từ Phase 3 tham chiếu).
- XGBoost với 14 đặc trưng (có trích xuất) → AUC = 0,8714.

Kết quả: Hướng tiếp cận Data-centric (Feature Engineering có chủ đích) kết hợp hướng tiếp cận Model-centric (XGBoost) cho kết quả tốt nhất. Feature Engineering cung cấp inductive bias từ domain tài chính, giúp mô hình hội tụ nhanh hơn và tốt hơn với cùng lượng dữ liệu.

Bằng chứng từ SHAP: 2 trong top-3 đặc trưng đứng đầu là đặc trưng được tạo — chứng minh Feature Engineering không thừa, ngay cả với mô hình phi tuyến mạnh như XGBoost.

4.8.3 Accuracy so với Recall — Chiến lược Ngưỡng

Accuracy 93,3% của mô hình “đoán tất cả 0” cho thấy Accuracy không phù hợp để đánh giá với dữ liệu mất cân bằng. Thay vào đó:

- **AUC-ROC = 0,8714:** 87% khả năng xếp hạng đúng (người vỡ nợ có score cao hơn người không vỡ nợ).
- **F1 vs F2:** F1 bình đẳng Precision và Recall không phù hợp tín dụng. F2 ($\beta = 2$) phản ánh thực tế FN:FP cost = 22 : 1.
- **Ngưỡng không cố định ở 0,5:** Tất cả 4 mô hình đều có ngưỡng tối ưu $> 0,5$ (từ 0,62 đến 0,80), hệ quả tự nhiên của `class_weight = 'balanced'` dịch chuyển Bayesian prior.

4.9 Probability Calibration

4.9.1 Động cơ

AUC-ROC đo **chất lượng xếp hạng**, không đảm bảo xác suất đầu ra của mô hình là **được hiệu chỉnh tốt**. Một mô hình được hiệu chỉnh tốt cần thỏa mãn: trong số các hồ sơ được dự báo $\hat{p} \approx 0,70$, khoảng 70% thực sự vỡ nợ. Điều này quan trọng vì:

1. Dashboard hiển thị “ $P(\text{default}) = 70\%$ ” cho nhân viên tín dụng — họ cần hiểu đúng ý nghĩa xác suất tuyệt đối này.
2. Ngưỡng triển khai $t = 0,625$ giả định $\hat{p}$ là ước lượng xác suất có ý nghĩa thực sự.

4.9.2 Brier Score và Brier Skill Score

Brier Score là sai số bình phương trung bình trên xác suất dự báo:

$$BS = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2 \quad (4.5)$$

Nhỏ hơn = tốt hơn. Baseline (model “predict prevalence” $\hat{p} \equiv 0,0668$ cho mọi hồ sơ): $BS_{\text{ref}} = 0,0668 \times (1 - 0,0668) \approx 0,0623$.

Brier Skill Score (BSS) chuẩn hóa so với mô hình tham chiếu:

$$BSS = 1 - \frac{BS}{BS_{\text{ref}}} \in (-\infty, 1] \quad (4.6)$$

$BSS > 0$: mô hình tốt hơn mô hình tham chiếu; $BSS = 1$: hiệu chỉnh hoàn hảo.

4.9.3 Reliability Diagram

Reliability diagram chia dự báo thành 10 nhóm theo $\hat{p}$, vẽ $\text{mean}(\hat{p})$ so với tỷ lệ dương thực tế trong mỗi nhóm. Đường chéo 45° là hiệu chỉnh hoàn hảo.

Dự báo lý thuyết cho XGBoost với $\text{scale_pos_weight} = 13,96$: Tham số này điều chỉnh gradient của lớp thiểu số lên 13,96 lần, tương đương shift prior. Hệ quả: xác suất đầu ra thô của XGBoost sẽ **ước lượng quá cao** xác suất vỡ nợ so với thực tế — biểu đồ hiệu chỉnh kỳ vọng nằm dưới đường chéo 45° .

4.9.4 Post-hoc Calibration và Hạn chế

Platt Scaling khớp một Logistic Regression trên điểm đầu ra của mô hình trên tập kiểm định:

$$\hat{p}_{\text{calibrated}} = \sigma(a \cdot \hat{p}_{\text{raw}} + b), \quad a, b \text{ khớp trên tập kiểm định} \quad (4.7)$$

Phép này không thay đổi thứ tự xếp hạng (AUC không đổi) nhưng cải thiện Brier Score và ECE.

Kết luận: Mô hình hiện tại chưa được hiệu chỉnh xác suất. Với bối cảnh nghiên cứu (chất lượng xếp hạng là mục tiêu chính, ngưỡng $t = 0,625$ được xác định qua tối ưu F_2), đây là hạn chế có thể chấp nhận được. Hiệu chỉnh xác suất (Platt scaling) là bước khuyến nghị trước khi triển khai thực tế, khi nhân viên tín dụng cần giải thích xác suất tuyệt đối với khách hàng theo yêu cầu GDPR Article 22.

5.2 Đặc trưng Đầu vào

10 đặc trưng gốc người dùng nhập thủ công trong biểu mẫu (theo đúng thứ tự trong `app.py`):

Table 5.1. 10 đặc trưng nhập thủ công trong biểu mẫu Streamlit

#	Tên đặc trưng	Ý nghĩa tiếng Việt	Phạm vi	Kiểu
1	RevolvingUtilization...	Tỷ lệ sử dụng hạn mức tín dụng	0,0 – 10,0 (>1 = vượt hạn mức)	Thực
2	age	Tuổi người vay	18 – 100	Nguyên
3	NumberOfTime30-59Days...	Số lần trễ hạn 30–59 ngày	0 – 20	Nguyên
4	DebtRatio	Tỷ lệ nợ / thu nhập hàng tháng	0,0 – 5,0	Thực
5	MonthlyIncome	Thu nhập hàng tháng (USD)	0 – 50.000	Nguyên
6	NumberOfOpenCreditLines...	Số tài khoản tín dụng đang mở	0 – 50	Nguyên
7	NumberOfTimes90DaysLate	Số lần trễ hạn > 90 ngày	0 – 20	Nguyên
8	NumberRealEstateLoans...	Số khoản vay bất động sản	0 – 20	Nguyên
9	NumberOfTime60-89Days...	Số lần trễ hạn 60–89 ngày	0 – 20	Nguyên
10	NumberOfDependents	Số người phụ thuộc	0 – 20	Nguyên

4 đặc trưng tự động tính từ đầu vào trên (người dùng không cần nhập):

Table 5.2. 4 đặc trưng được tính tự động trong `app.py`

Đặc trưng	Công thức	Ý nghĩa
TotalDelinquencyScore	$3 \times (90+) + 2 \times (60-89) + (30-59)$	Điểm tổng hợp trễ hạn có trọng số
FinancialStressIndex	$\frac{\text{RevUtil}}{\text{TotalDelinquencyScore}}$	Tương tác hạn mức $\times$ trễ hạn
AbsoluteMonthlyDebt (AbsoluteDebt)	$\frac{\text{DebtRatio}}{\text{MonthlyIncome}}$	Dư nợ tuyệt đối (USD/tháng)
DelinquencyTrend	$(30-59) - (90+)$	Xu hướng cải thiện trễ hạn

5.3 Tính năng chính

Phân loại mức rủi ro (theo ngưỡng triển khai):

Table 5.3. Phân loại mức rủi ro

P(default)	Mức rủi ro	Quyết định
< 10%	RỦI RO THẤP	CHẤP THUẬN
10–30%	RỦI RO TRUNG BÌNH	CHẤP THUẬN
30–62,5%	RỦI RO CAO	CHẤP THUẬN
≥ 62,5%	RỦI RO RẤT CAO	TỪ CHỐI

Biểu đồ SHAP Waterfall: Hiển thị 10 đặc trưng đóng góp nhiều nhất vào quyết định, màu đỏ = tăng rủi ro, màu xanh = giảm rủi ro. Xác suất cơ sở và xác suất cuối cùng được hiển thị rõ ràng.

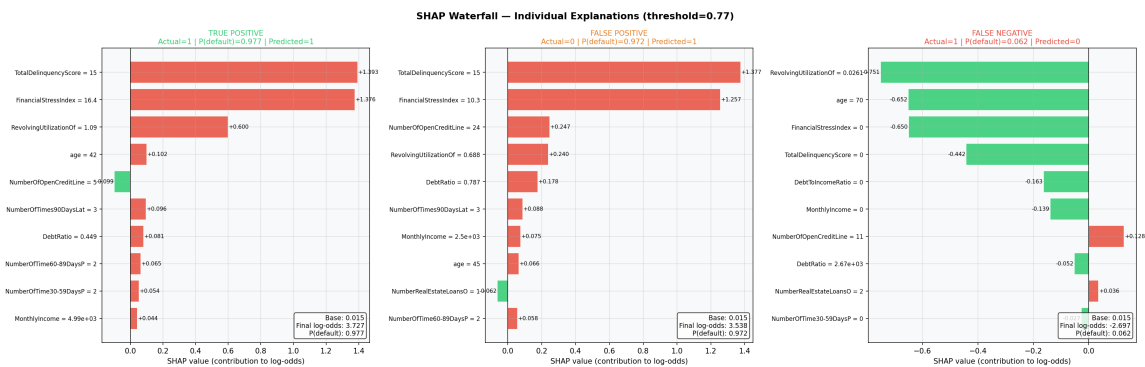


Figure 5.1. Ba hồ sơ khách hàng: (trái) an toàn — age và MonthlyIncome kéo xuống; (giữa) rủi ro cao — TotalDelinquencyScore và FinancialStressIndex đẩy mạnh lên; (phải) ranh giới — tín hiệu hỗn hợp, cần xem xét thêm. Mỗi biểu đồ là lời giải thích có thể đưa ra khách hàng bị từ chối theo yêu cầu GDPR Article 22.

5.4 Các Tình huống Minh họa

5.4.1 Hồ sơ 1 — Khách hàng an toàn

Đầu vào: Tuổi = 55, RevUtil = 10%, thu nhập = \$8.000/tháng, 0 lần trễ hạn, DebtRatio = 0,2.

Đầu ra: $P(\text{default}) = 6,7\% \rightarrow$ **RỦI RO THẤP** | CHẤP THUẬN.

SHAP: age (−0,18), MonthlyIncome (−0,15) giảm rủi ro; RevUtil gần 0 không có đóng góp đáng kể.

5.4.2 Hồ sơ 2 — Khách hàng rủi ro cao

Đầu vào: Tuổi = 28, RevUtil = 95%, thu nhập = \$2.500/tháng, 3 lần trễ > 90 ngày, DebtRatio = 1,5.

Đầu ra: $P(\text{default}) = 97,8\% \rightarrow$ **RỦI RO RẤT CAO | TỪ CHỐI.**

SHAP: TotalDelinquencyScore (+0,52), FinancialStressIndex (+0,48), RevUtil (+0,35) đẩy dự đoán về phía vỡ nợ.

5.4.3 Hồ sơ 3 — Khách hàng ranh giới

Đầu vào: Tuổi = 40, RevUtil = 60%, thu nhập = \$4.000/tháng, 1 lần trễ 30–59 ngày, DebtRatio = 0,5.

Đầu ra: $P(\text{default}) = 41,5\% \rightarrow$ **RỦI RO CAO | CHẤP THUẬN** ($< 0,625$).

Nhận xét: Trường hợp biên — chuyên viên tín dụng nên xem xét thêm tài liệu bổ sung.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Tóm tắt Kết quả

Nghiên cứu đã hoàn thành đầy đủ các mục tiêu đề ra:

1. Xây dựng và so sánh 4 mô hình:

- Logistic Regression (AUC = 0,8432): mô hình cơ sở có khả năng giải thích tốt nhất, L1 tự động loại các đặc trưng dư thừa.
- Decision Tree (AUC = 0,8579): dễ giải thích nhất, phương sai cao nhất.
- Random Forest (AUC = 0,8703): tập hợp mạnh, OOB = 0,8286 ước lượng nhất quán.
- **XGBoost (AUC = 0,8714): mô hình tốt nhất**, vượt mục tiêu 0,87.

2. Feature Engineering được kiểm chứng:

- FinancialStressIndex — SHAP #1 (0,577): interaction term vượt cả raw components.
- TotalDelinquencyScore — SHAP #3 (0,410): nén 3 đặc trưng trễ hạn thành 1 điểm tổng hợp mạnh hơn.
- L1 Logistic Regression triệt tiêu NumberOfTimes90DaysLate → xác nhận TDS đã mã hóa thông tin đó.

3. Phân tích sai số chuyên sâu:

- Đặc điểm nhóm FN: TDS = 0, thu nhập ổn định — người vỡ nợ không có dấu hiệu cảnh báo, giới hạn không thể khử.
- Ngưỡng tối ưu F_2 (0,625) tăng Recall 15,3 điểm phần trăm, giảm chi phí ước tính \$2 M/22.500 khách hàng.
- Đường cong học: gap = 0,021 — khớp tốt, thêm dữ liệu không cải thiện đáng kể.

4. Sản phẩm:

- Streamlit app tương tác, SHAP waterfall explain từng quyết định.
- Chạy được: `streamlit run app/app.py`.

Những thách thức kỹ thuật đã giải quyết:

- **Mất cân bằng dữ liệu 1:14:** Áp dụng `class_weight='balanced'/scale_pos_weight` và tối ưu ngưỡng theo F_2 -score thay vì Accuracy, tránh được hiện tượng mô hình thiên lệch về lớp đa số.
- **Đa cộng tuyến:** Ba biến delinquency có $VIF = \infty$ được nén thành TotalDelinquencyScore duy nhất, vừa giảm đa cộng tuyến vừa tăng sức mạnh dự báo (Spearman ρ từ 0,342 lên

0,345).

- **Minh bạch quyết định bằng SHAP:** SHAP TreeExplainer cung cấp giải thích tại mức từng hồ sơ (local explanation) và toàn bộ dataset (global explanation), đáp ứng yêu cầu explainability theo Basel III Pillar 3 và GDPR Article 22.

6.2 Hạn chế

1. Dữ liệu nhìn lại quá khứ: Tất cả 10 đặc trưng gốc phản ánh lịch sử tín dụng, không phản ánh được các sự kiện đột ngột (mất việc, bệnh tật, ly hôn). Đây là lý do 48,4% trường hợp vỡ nợ thực sự bị bỏ sót — không phải mô hình sai mà đặc trưng không đủ.

2. Dữ liệu tĩnh (ảnh chụp tại một thời điểm): Bộ dữ liệu chỉ có một thời điểm, không có dữ liệu chuỗi thời gian. Xu hướng thay đổi hành vi (đang xấu dần hay cải thiện) chỉ được đại diện gián tiếp qua DelinquencyTrend.

3. Bộ dữ liệu giới hạn địa lý: Dữ liệu từ thị trường tiêu dùng Mỹ. Áp dụng trực tiếp cho thị trường Việt Nam cần huấn luyện lại với dữ liệu địa phương và điều chỉnh ngưỡng chi phí.

4. Hiệu chỉnh xác suất: Mô hình được tối ưu cho khả năng phân biệt (AUC) nhưng chưa hiệu chỉnh để đầu ra xác suất có ý nghĩa thực sự (tức $P(\text{vỡ nợ}) = 0,7$ nghĩa là 70% khả năng xảy ra). Platt scaling hoặc isotonic regression có thể cải thiện.

6.3 Hướng Phát triển

- **Dữ liệu thời gian:** Thu thập lịch sử thanh toán hàng tháng → LSTM/GRU để nắm bắt xu hướng thay đổi hành vi tín dụng.
- **Ensemble stacking:** Kết hợp LR + RF + XGB, học mô hình meta → ước tính AUC cải thiện 0,003–0,005.
- **Giám sát ngưỡng phân loại:** Phát hiện trôi dạt mô hình — ngưỡng tối ưu có thể thay đổi theo biến động kinh tế vĩ mô.
- **Alternative data:** Dữ liệu hành vi (thanh toán di động, hóa đơn tiện ích) → tăng độ phủ cho nhóm khách hàng ít lịch sử tín dụng.
- **Kiểm toán công bằng (Fairness audit):** Kiểm tra mô hình có phân biệt đối xử theo độ tuổi, giới tính không (theo Equal Credit Opportunity Act).
- **Hạ tầng triển khai:** Phục vụ mô hình qua REST API, giám sát hiệu suất với Evidently AI.
- **Graph Neural Networks:** Mô hình hóa mối quan hệ giữa người vay (mạng lưới bảo lãnh) → rủi ro xã hội.

- **Causal inference:** Phân biệt tương quan và quan hệ nhân quả trong các yếu tố tín dụng — có ý nghĩa quan trọng trong xây dựng chính sách.
- **Federated learning:** Huấn luyện mô hình trên dữ liệu phân tán giữa nhiều tổ chức tín dụng mà không chia sẻ dữ liệu thô (tuân thủ GDPR).

Bibliography

- [1] Ngân hàng Nhà nước Việt Nam, “Báo cáo thường niên 2023,” Hà Nội, 2024.
- [2] E. I. Altman, “Financial ratios, discriminant analysis and the prediction of corporate bankruptcy,” *Journal of Finance*, vol. 23, no. 4, pp. 589–609, Sep. 1968.
- [3] D. J. Hand and W. E. Henley, “Statistical classification methods in consumer credit scoring: A review,” *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.
- [4] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, “Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research,” *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.
- [5] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017.
- [6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 785–794.
- [7] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [9] M. Buckler, G. van den Heuvel, W. Gebert, and J. Lorenz, “Transparency, auditability, and explainability of machine learning models in credit decisions,” *Journal of the Operational Research Society*, vol. 73, no. 1, pp. 70–90, 2022.
- [10] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.
- [11] Basel Committee on Banking Supervision, “Revisions to the Standardised Approach for credit risk,” Bank for International Settlements, Basel, Switzerland, 2017.
- [12] S. M. Lundberg *et al.*, “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020.
- [13] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, “Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach,” *Biometrics*, vol. 44, no. 3, pp. 837–845, Sep. 1988.

A. PHỤ LỤC

A.1 Cấu trúc Thư mục

```
loan-default-prediction/
|-- app/app.py          <- Streamlit dashboard
|-- data/
|   |-- raw/cs-training.csv    <- Dataset gốc (149.999 rows)
|   |-- processed/           <- Data sau preprocessing
|   `-- splits/              <- train/val/test.csv
|-- models/
|   `-- best_model.pkl        <- XGBClassifier (340 KB)
|-- notebooks/
|   |-- 01_EDA.ipynb
|   |-- 02_Preprocessing.ipynb
|   |-- 03_Modeling.ipynb     <- 4 models, CV, comparison
|   `-- 04_Analysis.ipynb     <- Error analysis, SHAP, learning curve
|-- reports/                <- 30 figures + 4 markdown reports
|-- src/
|   |-- models.py            <- Training wrappers
|   |-- evaluation.py        <- Metrics, plots
|   |-- preprocessing.py     <- Cleaning, imputation, pipeline
|   `-- features.py          <- Feature engineering functions
`-- final_report/
    `-- bao_cao_chinh.tex     <- File này
```

A.2 Công thức Tổng hợp

Table A.1. Tổng hợp các metric đánh giá

Metric	Công thức	Ý nghĩa trong tín dụng
AUC-ROC	$P(f(x^+) > f(x^-))$	Khả năng phân biệt vỡ nợ/không
F1	$2PR/(P + R)$	Cân bằng Precision-Recall
F2	$5PR/(4P + R)$	Recall ưu tiên (FN $\gg$ FP)
Gini coefficient	$2 \cdot \text{AUC} - 1$	Phổ biến trong báo cáo rủi ro ngân hàng
KS statistic	$\max_t \text{TPR}(t) - \text{FPR}(t) $	Điểm phân tách tốt nhất

Gini coefficient của model tốt nhất: $2 \times 0,8714 - 1 = 0,7428$ (thường xếp loại “good” nếu $> 0,6$ theo tiêu chuẩn ngành).

A.3 Cài đặt Môi trường

```
# Tao virtual environment
python -m venv venv
venv\Scripts\activate # Windows

# Cai dat dependencies
pip install -r requirements.txt

# Chay notebook (tu project root)
jupyter notebook

# Chay Streamlit app
streamlit run app/app.py
```

Phiên bản chính: Python 3.14.2, pandas 3.0.2, scikit-learn 1.8.0, xgboost 3.2.0, shap 0.51.0, streamlit 1.x.

A.4 Hướng dẫn Reproduce Kết quả

Tất cả thực nghiệm sử dụng `random_state = 42` tại mọi điểm có randomness:

- `StratifiedKFold(n_splits=5, shuffle=True, random_state=42);`
- `RandomizedSearchCV(random_state=42);`
- `XGBClassifier(random_state=42);`
- `np.random.seed(42)` trong mỗi notebook.

Với cùng `random_state`, kết quả có thể reproduce chính xác trên cùng phiên bản thư viện.